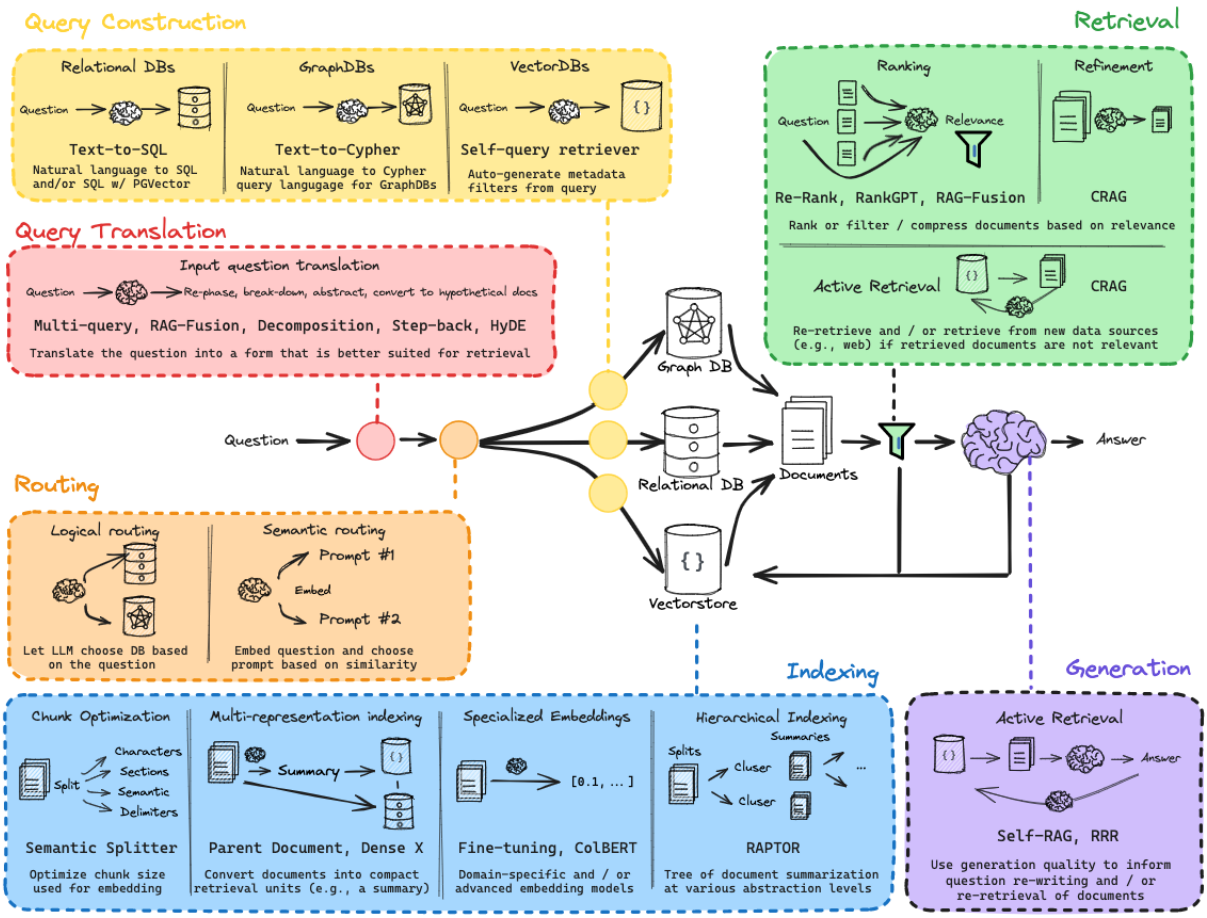
RAG and Agentic Design Pattern



Multi Query Automatically the user's input into multiple distinct valuation tackling the problem from different linguistic angles.

- Expansion - When user submits a query, it is sent to a smaller faster LLM instructing it to generate N variation of the question.
- Parallel Retrieval - All queries are sent parallelly to retrieve document from the vector datbase.
- De-duplication and union
- Generation

RAG Fusion introduces a clever scoring algorithm called Reciprocal Rank Fusion(RRF) to re-organise and rank those documents befor feeding them to LLM.

$$RRF(d) = \sum_{q \in Q} \frac{1}{k + r_q(d)}$$

Q : Set if all Query Variation
d : Specific rank of document 'd' in result of query 'q'
k : constant smoothing factor

Query Decomposition break the single multi step question down to smaller distinct sub-questions. Each sub-question targets a different piece of missing information

- Parallel decomposition (interdependent topics)
- Sequential decomposition (state based RAG)

Step-back prompting uses and LLM to "take a step back" and generate broad high level abstract question that capture the overacting concept of foundational principle behind the user request.

- Abstracting the query - Prompt passed to small fast LLM to identify the core principal and write a high level generalised version of the question.
- Dual vector search - Run two separate query against vector database in parallel.
- Prompt augmentation and context stitching - merge result of both searches.

- Generation

HyDE (Hypothetical Document Embedding) the process intercepts the user's prompt before it reaches the vector database.

- Hallucinate an answer - System passes user's query to instruction following LLM. The LLM will generate a synthetic hypothetical document.
- Encode the fake document
- Retrieve the real document- The fake document vector is used to do similarity search against the vector database.

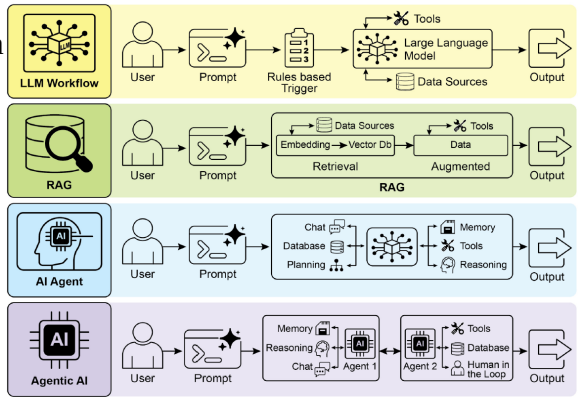
Routing The router analysis your query and direct it to the most appropriate model, database or tool to handle specific request.

- Logical router uses if/else statements RegEx or simple string matching to look for exact words.
- Semantic - uses lightweight embedding model. Create a predefined list of 'topics' and embed them into vector space. For your query, check the topic closest to.
- LLM based routing - uses fast inexpensive LLM to act as a judge.

Indexing

Chunk optimization if chunks are too small, you lose vital surrounding context. If they are two large, they contain two many distinct ideas.
Semantic splitter – reads a text, look for shifts in the meaning to decide where to break chunks.
Multi representation Indexing system takes a large raw chunk of text, LLM distils it into high level summary. When retrieved , databases uses a pointer to full original text and passes that to generation LLM.
ColBERT generate individual embeding vector for every single word (token) in query and every single word in document. During retrieval, it computes alignment between all the words in the query and all the words in the document using MaxSim operator.
Hierarchical Indexing RAPTOR (Recursive Abstractive Processing for Tree Organised Retrieval) start by clustering related low level text chunks using an embeding model. LLM write the summary of each cluster. These summaries are themselves embeded and clustered again to create even higher level summary. Recursive process repeat until it forms multi layer tree structure

LLM Workflow: Represents a standard process where a user's prompt passes through rule-based triggers before being processed by a Large Language Model (LLM), which can optionally draw on external tools and data sources.
RAG: Stands for Retrieval-Augmented Generation. It highlights a system where a prompt is augmented with specific, relevant data retrieved from a vector database before the final output is generated.
AI Agent: Depicts a more autonomous, single-entity system. The core AI has direct, multi-directional access to various cognitive and functional modules—like reasoning, memory, planning, and tools—allowing it to handle complex, multi-step tasks.
Agentic AI: Illustrates a collaborative, multi-agent ecosystem. Instead of one central AI, multiple specialized agents work together.



Prompt Chaining

Prompt chaining is a divide-and-conquer strategy that breaks complex tasks into a sequence of smaller, manageable sub-problems. The output generated by one prompt is specifically used as the input for the subsequent prompt in the chain. This approach introduces modularity, making it easier to debug individual steps and optimize outputs. It serves as a foundational technique for building sophisticated AI agents capable of multi-step reasoning, planning, and autonomous action.
Complex, single prompts often cause large language models (LLMs) to struggle, leading to instruction neglect, contextual drift, error propagation, and hallucination. Sequential decomposition resolves this by isolating tasks—such as summarization, extraction, and drafting—into separate steps with distinct model roles. This workflow significantly reduces the model's cognitive load, leading to more accurate and reliable final outputs.
The success of a prompt chain relies heavily on the integrity of the data passed between steps. To prevent ambiguity or faulty inputs, it is crucial to require structured output formats like JSON or XML. Structured formats ensure data is machine-readable, precisely parsed, and easily inserted into the next step.

Practical Applications

- Information Processing & Query Answering: Extracting entities, searching databases, and synthesizing data from multiple sources to answer complex questions.
- Data Extraction & Transformation: Iteratively refining unstructured text (like invoices or OCR outputs) into validated, structured data.
- Content Generation: Breaking writing tasks into sequential phases like ideation, outlining, drafting, and revision.
- Conversational Agents: Maintaining context across multi-turn dialogues by using conversation history to update the agent's state.
- Code Generation: Decomposing software development into outlining, drafting, identifying errors, refining, and documenting.